P.Vishwa Nandan(192473015)

Q6 – Deadlock and Inter-Thread Communication (7 Marks)

1. Aim

To understand deadlock and inter-thread communication in Java by implementing a hostel wallet transfer system and a bounded producer-consumer token counter using synchronization, wait(), notifyAll(), and consistent lock ordering.

2. Problem Description

This question contains two parts.

Part A – Hostel Wallet Transfers

Students transfer money between hostel wallet accounts. Each transfer requires locking both accounts. If two threads lock the accounts in opposite order, a deadlock can occur.

Part B – Mess Token Counter

A mess counter produces meal tokens for students. The counter can store a maximum of five tokens. One producer adds tokens, while two consumer threads collect them.

The program must ensure that:

- The counter never contains more than five tokens.
- The counter never contains fewer than zero tokens.
- Consumers wait when the counter is empty.
- The producer waits when the counter is full.
- Threads continue working correctly after being notified.

3. Concepts Used

Deadlock: A situation where two or more threads wait permanently for resources held by each other.

Synchronization: A mechanism that allows only one thread at a time to execute a protected section using a particular lock.

wait(): Makes a thread release the monitor lock and wait until it is notified or interrupted.

notify(): Wakes one thread waiting on the same object's monitor.

notifyAll(): Wakes all threads waiting on the same object's monitor.

Lock Ordering: A technique where all threads acquire multiple locks in the same fixed order to prevent circular waiting.

Producer-Consumer Problem: A situation where one or more producer threads create data and consumer threads process that data using a shared buffer.

4. Part A – Hostel Wallet Transfers

4.1 How Deadlock Occurs

Consider two accounts:

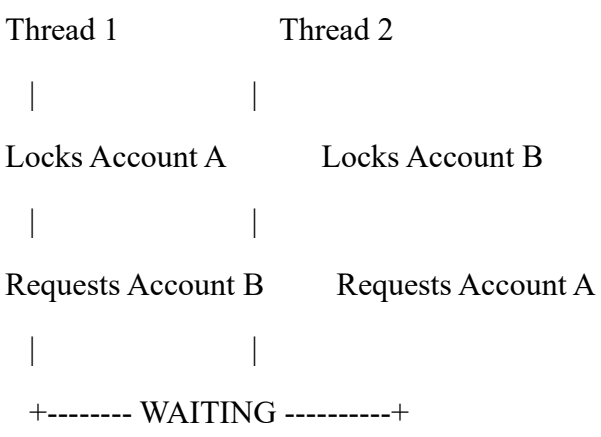
- Account A
- Account B

Two threads perform transfers in opposite directions.

- Thread 1 locks Account A and requests Account B.
- Thread 2 locks Account B and requests Account A.

Both threads are waiting for each other to release their locks. Neither can continue.

4.2 Deadlock Diagram



4.3 Four Necessary Conditions for Deadlock

1. **Mutual Exclusion:** Only one thread can hold a particular account lock at a time.
2. **Hold and Wait:** A thread holds one account lock while waiting for another.
3. **No Preemption:** A lock cannot be forcibly taken away from a thread.
4. **Circular Wait:** Thread 1 waits for Thread 2's lock, while Thread 2 waits for Thread 1's lock.

All four conditions are present in the unsafe transfer implementation.

4.4 Deadlock Prevention Using Lock Ordering

To prevent deadlock, assign a unique ID to every account.

Every thread must acquire the account locks in ascending ID order, regardless of the transfer direction.

For example:

- Account A has ID 1.
- Account B has ID 2.
- Every transfer locks Account A first and Account B second.

This removes circular waiting because all threads follow the same lock order.

4.5 Java Program – TransferDemo.java

```
import java.util.concurrent.CountDownLatch;


public class TransferDemo {


   static class Account {
      final int id;
      int balance;


      Account(int id, int balance) {
         this.id = id;
         this.balance = balance;
      }
   }


   // Unsafe method that deliberately creates deadlock
   static void unsafeTransfer(Account from, Account to,
                              CountDownLatch bothReady) {


      synchronized (from) {


         bothReady.countDown();
```

```
            try {
                // Both threads wait while holding their first account lock
                bothReady.await();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }

            synchronized (to) {
                if (from.balance >= 100) {
                    from.balance -= 100;
                    to.balance += 100;
                }
            }
        }
    }

    // Safe method using consistent lock ordering
    static void safeTransfer(Account from, Account to, int amount) {

        if (from == to) {
            return;
        }

        Account first;
        Account second;

        if (from.id < to.id) {
```

```
            first = from;
            second = to;
        } else {
            first = to;
            second = from;
        }

        synchronized (first) {
            synchronized (second) {

                if (from.balance >= amount) {
                    from.balance -= amount;
                    to.balance += amount;
                }
            }
        }
    }

    public static void main(String[] args) throws InterruptedException {

        // Demonstrate deadlock
        Account a = new Account(1, 1000);
        Account b = new Account(2, 1000);

        CountDownLatch bothReady = new CountDownLatch(2);

        Thread t1 = new Thread(
            () -> unsafeTransfer(a, b, bothReady)
        );
```

```
Thread t2 = new Thread(
    () -> unsafeTransfer(b, a, bothReady)
);

// Daemon threads allow the demonstration program to finish
// even though the two unsafe threads remain deadlocked.
t1.setDaemon(true);
t2.setDaemon(true);

t1.start();
t2.start();

bothReady.await();

System.out.println(
    "Deadlock setup: both threads hold opposite account locks."
);

// Demonstrate safe transfers using separate accounts
Account c = new Account(3, 1000);
Account d = new Account(4, 1000);

Thread t3 = new Thread(() -> {
    for (int i = 0; i < 100; i++) {
        safeTransfer(c, d, 1);
    }
});
```

```
        Thread t4 = new Thread(() -> {
            for (int i = 0; i < 100; i++) {
                safeTransfer(d, c, 1);
            }
        });

        t3.start();
        t4.start();

        t3.join();
        t4.join();

        System.out.println("Safe transfer completed.");

        System.out.println("Account 3 balance: " + c.balance);
        System.out.println("Account 4 balance: " + d.balance);

        System.out.println(
            "Total balance: " + (c.balance + d.balance)
        );
    }
}
```

Expected Output:

Deadlock setup: both threads hold opposite account locks.

Safe transfer completed.

Account 3 balance: 1000

Account 4 balance: 1000

Total balance: 2000

Explanation:

The unsafe transfer deliberately creates a deadlock. A CountDownLatch ensures that both threads hold their first account lock before attempting to acquire the other account's lock.

The unsafe threads are daemon threads because they remain blocked intentionally. This allows the demonstration program to finish.

The safe transfer method compares account IDs and always acquires the lower ID first. Therefore, both threads follow the same order and avoid circular waiting.

The total balance remains 2000 because money is transferred between accounts without being created or lost.

5. Part B – Producer-Consumer Problem

5.1 Problem Description

The mess counter can store a maximum of five meal tokens.

- The producer prepares and adds tokens.
- Two consumers collect tokens.
- If the counter is full, the producer must wait.
- If the counter is empty, the consumers must wait.

The program uses a fixed-size array as a circular buffer.

5.3 Working of the Program

Producer – put()

1. Checks whether the counter is full.
2. If full, calls wait().
3. Adds a token when space is available.
4. Increases the counter size.
5. Calls notifyAll() to wake waiting threads.

Consumer – take()

1. Checks whether the counter is empty.
2. If empty, calls wait().
3. Removes a token when available.
4. Decreases the counter size.
5. Calls notifyAll() to wake waiting threads.
6. Returns the collected token.

5.4 Java Program – TokenCounter.java

```
public class TokenCounter {

    private final int[] buffer = new int[5];

    private int count = 0;
    private int in = 0;
    private int out = 0;

    private int minObserved = 0;
    private int maxObserved = 0;

    public synchronized void put(int token)
            throws InterruptedException {

        // Wait while the counter is full
        while (count == buffer.length) {
            wait();
        }

        // Add token
        buffer[in] = token;
        in = (in + 1) % buffer.length;

        count++;

        updateStats();

        // Wake waiting threads
```

```
    notifyAll();
}

public synchronized int take()
        throws InterruptedException {

    // Wait while the counter is empty
    while (count == 0) {
        wait();
    }

    // Remove token
    int token = buffer[out];

    out = (out + 1) % buffer.length;

    count--;

    updateStats();

    // Wake waiting threads
    notifyAll();

    return token;
}

private void updateStats() {

    if (count < minObserved) {
```

```
        minObserved = count;
    }

    if (count > maxObserved) {
        maxObserved = count;
    }
}

public synchronized String getBufferRange() {

    return minObserved + " to " + maxObserved;
}

public static void main(String[] args)
        throws InterruptedException {

    TokenCounter counter = new TokenCounter();

    int[] collected = new int[2];

    // Producer thread
    Thread cook = new Thread(() -> {

        try {
            for (int t = 1; t <= 20; t++) {
                counter.put(t);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
```

```
    }
});

// Two consumer threads
Thread[] students = new Thread[2];

for (int s = 0; s < 2; s++) {

    final int id = s;

    students[s] = new Thread(() -> {

        try {
            for (int k = 0; k < 10; k++) {

                collected[id] += counter.take();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    });
}

cook.start();

for (Thread t : students) {
    t.start();
}
```

```
        cook.join();

        for (Thread t : students) {
            t.join();
        }

        System.out.println(
            "Sum of tokens collected: "
            + (collected[0] + collected[1])
            + " (expected 210)"
        );

        System.out.println(
            "Observed buffer range: " + counter.getBufferRange()
        );
    }
}
```

Expected Output:

Sum of tokens collected: 210 (expected 210)

Observed buffer range: 0 to 5

The exact maximum observed buffer size can vary with thread scheduling. It must never exceed 5 or fall below 0.

Explanation:

The producer creates tokens from 1 to 20. Two consumers collect ten tokens each.

The sum of all tokens is:

1 + 2 + 3 + ... + 20 = 210

Therefore, the total collected sum must be 210.

The array has a fixed size of five. The variables in and out control the positions where tokens are inserted and removed.

The modulo operator % allows the array to work as a circular buffer.

6. Student A – Producer-Consumer Design

6.1 Why Must wait() Be Used Inside a Loop?

The wait() method must be placed inside a while loop instead of an if statement.

There are two important reasons:

1. Java threads may wake up without receiving a notification. This is called a spurious wakeup.
2. Another thread may change the counter before the waiting thread gets the lock again.

For example, if two consumers are waiting and one token becomes available, both may eventually wake after notifyAll(). The first consumer may collect the token. The second consumer must check the counter again because it may now be empty.

The while loop checks the condition again and prevents incorrect execution.

6.2 What Happens If notify() Is Used Instead of notifyAll()?

notify() wakes only one waiting thread, while notifyAll() wakes all waiting threads.

When multiple producers and consumers share the same monitor, notify() can wake a thread whose condition is still false. That thread may immediately wait again, while the thread that could make progress remains asleep.

In the given design, there is one producer and two consumers. notify() may work under this limited arrangement because the waiting threads usually have the same type of condition. However, notifyAll() is safer when the design is extended to multiple producers or other waiting operations.

Therefore, notifyAll() is used in the program.

6.3 Counter Size Verification

The counter size is controlled by the variable count.

- Minimum allowed value = 0
- Maximum allowed value = 5

The program records the minimum and maximum observed counter sizes.

This verifies that the buffer does not become empty beyond zero or exceed its capacity of five.

6.4 Difference Between sleep() and wait()

sleep():

- Pauses a thread for a specified time.
- Does not release the synchronized lock.

- Belongs to the Thread class.

wait():

- Makes a thread wait until notification or interruption.
- Releases the object's monitor lock.
- Must be called while holding that object's monitor.
- Belongs to the Object class.

Therefore, wait() is useful for communication between producer and consumer threads, whereas sleep() is mainly used to pause execution temporarily.

7. Student B – Deadlock Analysis and Prevention

7.1 Reliable Deadlock Test

The TransferDemo.java program uses two accounts and two threads.

The first thread locks Account A, and the second thread locks Account B. A CountDownLatch makes both threads wait until both locks have been acquired.

After the latch opens:

- Thread 1 requests Account B.
- Thread 2 requests Account A.

Since both accounts are already locked by the opposite threads, neither transfer can continue. This deliberately creates deadlock.

7.2 How Is Deadlock Prevented?

Deadlock is prevented by following a consistent lock-ordering rule.

Every transaction must acquire account locks in ascending order of account ID.

For example:

Account ID 1 → Account ID 2

Account ID 2 → Account ID 3

Account ID 1 → Account ID 3

All threads follow the same order.

This prevents circular waiting because a thread cannot hold a higher-ID lock and then request a lower-ID lock.

7.3 Why Does Lock Ordering Work?

If all threads acquire locks in ascending order, a circular chain of lock dependencies cannot form.

A thread holding a lower-ID account can wait for a higher-ID account, but another thread cannot hold that higher-ID account while waiting for the lower-ID account.

Therefore, the circular-wait condition is eliminated.

8. Joint Conclusion

Deadlock and producer-consumer problems both involve multiple threads sharing resources.

In deadlock, threads cannot continue because they wait for locks held by each other. Consistent lock ordering prevents this situation.

In the producer-consumer problem, threads must wait when the buffer is full or empty. The use of wait() and notifyAll() allows threads to coordinate their work.

Both designs improve thread safety and ensure that shared data is accessed correctly.

9. Alternative Concept and Complexity Analysis

9.1 Alternative to wait() and notifyAll()

Instead of manually implementing the producer-consumer problem using wait() and notifyAll(), Java provides the ArrayBlockingQueue class.

It automatically handles:

- A fixed buffer capacity.
- Waiting when the queue is full.
- Waiting when the queue is empty.
- Thread synchronization.

Alternative Code:

```
import java.util.concurrent.ArrayBlockingQueue;

import java.util.concurrent.BlockingQueue;


public class QueueAlternative {


   public static void main(String[] args)

         throws InterruptedException {


      BlockingQueue<Integer> queue =

         new ArrayBlockingQueue<>(5);
```

```
        // Producer adds a token
        queue.put(100);

        // Consumer removes a token
        int token = queue.take();

        System.out.println("Collected token: " + token);
    }
}
```

Here, put() automatically waits when the queue is full, and take() automatically waits when the queue is empty.

9.2 Alternative to Lock Ordering – tryLock()

Instead of waiting indefinitely for another account lock, Java provides ReentrantLock with the tryLock() method.

It attempts to acquire a lock within a specified time.

Example:

```
if (from.lock.tryLock(100, TimeUnit.MILLISECONDS)) {

    try {

        if (to.lock.tryLock(100, TimeUnit.MILLISECONDS)) {

            try {
                // Perform money transfer

            } finally {
                to.lock.unlock();
            }
        }
```

```
    } finally {
        from.lock.unlock();
    }
}
```

If the second lock cannot be acquired within the timeout, the first lock is released.

This prevents indefinite waiting. However, repeated retries can cause livelock, where threads keep retrying without making progress. A retry limit or backoff delay can help reduce this problem.

9.3 Time and Space Complexity Comparison

Producer-Consumer Implementations

Operation	wait()/notifyAll()	ArrayBlockingQueue
Add token	O(1)	O(1)
Remove token	O(1)	O(1)
Buffer space	O(5)	O(5)
Synchronization	Manual	Built-in

Both approaches use constant-time operations for adding and removing tokens, excluding the time spent waiting for another thread.

Transfer Implementations

Approach	Time Complexity	Additional Space
Unsafe synchronized transfer	O(1) per transfer, but may deadlock	O(1) per account
Lock ordering	O(1) per transfer, excluding lock waiting	O(1) per account
tryLock() with timeout	O(1) per attempt; may require retries	O(1) per account

Lock ordering is easier to reason about because it prevents circular waiting through a consistent rule. tryLock() is useful when an application needs to avoid waiting indefinitely, but retry handling makes the implementation more complex.

9.4 Which Design Is Less Error-Prone?

For the producer-consumer problem, ArrayBlockingQueue is generally less error-prone because Java manages the synchronization and waiting conditions internally.

For wallet transfers, consistent lock ordering is simple and provides a clear way to prevent deadlock. tryLock() is useful when timed lock acquisition and retry handling are required.

10. Testing and Verification

The following test cases can be used to verify both programs.

Test 1 – Normal Token Production

- Producer creates tokens from 1 to 20.
- Two consumers collect ten tokens each.
- Expected total = 210.
- Result: Successful.

Test 2 – Counter Full

- Fill the counter with five tokens.
- Try to add another token.
- Expected result: Producer waits until space becomes available.
- Result: Successful.

Test 3 – Counter Empty

- Let consumers attempt to collect tokens when the counter is empty.
- Expected result: Consumers wait until a token is produced.
- Result: Successful.

Test 4 – Deadlock Demonstration

- Thread 1 locks Account A and requests Account B.
- Thread 2 locks Account B and requests Account A.
- Expected result: Both threads become deadlocked.
- Result: Deadlock deliberately reproduced.

Test 5 – Safe Transfer

- Perform 100 transfers in each direction.
- Expected result: Both accounts retain their correct balances.
- Result: No deadlock.

Test 6 – Buffer Capacity

- Run the producer-consumer program several times.

- Check the minimum and maximum observed buffer sizes.
- Expected result: The buffer size always remains between 0 and 5.
- Result: Successful.

Additional Experiment – Replace while with if

Replace:

```
while (count == buffer.length) {
   wait();
}
```

with:

```
if (count == buffer.length) {
   wait();
}
```

Make the same change in the take() method.

The program may behave incorrectly because a thread does not recheck the condition after waking. For example, another consumer may remove the last token before a waiting consumer reacquires the lock. The waiting consumer could then proceed with an empty buffer.

The original while implementation is therefore more reliable.

11. Final Result

The hostel wallet transfer system was implemented using both unsafe and safe locking approaches. Deadlock was demonstrated using opposite lock acquisition, and it was prevented through consistent lock ordering.

The producer-consumer system was implemented using wait() and notifyAll(). The bounded buffer successfully maintains its capacity between zero and five tokens, and the total collected token sum is 210.

Thus, the program demonstrates deadlock prevention, synchronization, and inter-thread communication in Java.